

Web Science

Assignment – 5

Institute of Web Science and Technologies

Universität Koblenz-Landau

Group name: FOWLER

Name: Jeevan Saraf

jeevansaraf@uni-koblenz.de

Matriculation no: **222100445**

Name: Parth Chavda

parthchavda@uni-koblenz.de

Matriculation no: **222100495**

Name: Prajna Shetty

prajnashetty73@uni-koblenz.de

Matriculation no: **222100479**

Name: Ritika Bhandari

rbhandari@uni-koblenz.de

Matriculation no: **222100563**

1 Simple Crawler

Your task in this exercise is to "crawl" all the pages of <http://quotes.toscrape.com>. Please keep in mind the following points to complete this assignment :

- Please have a look at the next exercise before you start this one.
- Your crawler should crawl all the quotes, their author (if mentioned), the tags associated with the quotes and form a comprehensive dataframe.
- Your crawling should stop when you encounter a page that no longer contains any quotes.
- You must provide the time taken crawl the above mentioned website.

Answer:

```
import requests
import re
from bs4 import BeautifulSoup
import pandas as pd

page = '/page/1/'
data = []

while True:
    url = "http://quotes.toscrape.com" + page
    f = requests.get(url)
    soup = BeautifulSoup(f.content, 'html.parser')
    a = soup.find_all('div',{'class':'quote'})

    for el in a:
        quote = el.find('span',{'class':'text'}).text
        author = el.find('small',{'class':'author'}).text
        tags = []
        for tag in el.find('div',{'class':'tags'}).find_all('a'):
```

```

temp = tag.text
tags.append(temp)
tags = ','.join(str(item) for item in tags)
page_no = re.findall("\d+",page)[0]
temp_tuple = (page_no,quote,author,tags)
data.append(temp_tuple)

```

```

if soup.find("li",{ 'class':'next'}) != None:
    page = soup.find("li",{ 'class':'next'}).find('a').get("href")
else:
    break

```

```

df = pd.DataFrame(data = data, columns = ['Page','Quote','Author','Tags'])
df

```

2 Descriptive Statistics

If you have successfully completed the first exercise of this assignment, then please provide the following details. You may have to tweak your code in the above exercise for some of the results.

1. What is the total count of valid pages in the aforementioned website that you could crawl?

```

In [2]: df['Page'] = df['Page'].astype(int)
df['Page'].max()

```

```

Out[2]: 10

```

2. Provide a frequency distribution of the number of quotes per author

```

In [3]: no_quote_per_author = df.groupby('Author')['Quote'].count()
no_quote_per_author = no_quote_per_author.to_frame()
no_quote_per_author.reset_index(inplace=True)
no_quote_per_author

```

Out[3]:

	Author	Quote
0	Albert Einstein	10
1	Alexandre Dumas fils	1
2	Alfred Tennyson	1
3	Allen Saunders	1
4	André Gide	1
5	Ayn Rand	1
6	Bob Marley	3
7	C.S. Lewis	5
8	Charles Bukowski	2
9	Charles M. Schulz	1
10	Douglas Adams	1
11	Dr. Seuss	6
12	E.E. Cummings	1
13	Eleanor Roosevelt	2
14	Elie Wiesel	1
15	Ernest Hemingway	2
16	Friedrich Nietzsche	1
17	Garrison Keillor	1
18	George Bernard Shaw	1

19	George Carlin	1
20	George Eliot	1
21	George R.R. Martin	2
22	Harper Lee	1
23	Haruki Murakami	1
24	Helen Keller	1
25	J.D. Salinger	1
26	J.K. Rowling	9
27	J.M. Barrie	1
28	J.R.R. Tolkien	1
29	James Baldwin	1
30	Jane Austen	5
31	Jim Henson	1
32	Jimi Hendrix	1
33	John Lennon	1
34	Jorge Luis Borges	1
35	Khaled Hosseini	1
36	Madeleine L'Engle	1
37	Marilyn Monroe	7
38	Mark Twain	6

39	Martin Luther King Jr.	1
40	Mother Teresa	2
41	Pablo Neruda	1
42	Ralph Waldo Emerson	2
43	Stephenie Meyer	1
44	Steve Martin	1
45	Suzanne Collins	2
46	Terry Pratchett	1
47	Thomas A. Edison	1
48	W.C. Fields	1
49	William Nicholson	1

3. Provide a frequency distribution of the tags

```
In [4]: no_of_tags = {}
        for el in df['Tags']:
            temp = el.split(',')
            for t in temp:
                if t not in no_of_tags:
                    no_of_tags[t] = 1
                else:
                    no_of_tags[t] += 1
no_of_tags
```

```
Out[4]: {'change': 1,
         'deep-thoughts': 1,
         'thinking': 2,
         'world': 1,
         'abilities': 1,
         'choices': 1,
         'inspirational': 13,
         'life': 13,
         'live': 1,
         'miracle': 1,
         'miracles': 1,
         'aliteracy': 1,
         'books': 11,
         'classic': 2,
         'humor': 12,
         'be-yourself': 1,
         'adulthood': 1,
```

4. Plot the average length of quotes (no. of words) per author

```
In [15]: import matplotlib.pyplot as plt

a = df.groupby(['Author'])['Quote'].apply(' '.join).reset_index()
word_len = []
x = list(a.Quote.str.split(' '))
for el in x:
    word_len.append(len(el))

a['word_len'] = word_len
a['no_quote_per_author'] = a['no_quote_per_author']
a['avg_len'] = a['word_len']/a['no_quote_per_author']
display(a)
fig = plt.figure()
ax = fig.add_axes([0,0,1,1])
Author = a['Author']
Avg_length_quote = a['avg_len']
ax.bar(Author,Avg_length_quote)
plt.xticks(rotation = 90)
plt.show()
```

